[CENTRO UNIVERSITARIO UNI GOIÁS]

CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

SISTEMA DE GESTÃO OLÍMPICA: DESENVOLVIMENTO DE UMA APLICAÇÃO WEB PARA EQUIPES, ATLETAS E RESULTADOS

GUILHERME TOMAZ

[GOIÂNIA]
2026

RESUMO

Este trabalho apresenta o desenvolvimento de um sistema web acadêmico destinado à gestão de informações olímpicas. A solução centraliza modalidades, equipes, atletas, resultados e usuários, com autenticação e controle de acesso por perfis. A documentação organiza o levantamento de requisitos, os requisitos funcionais e não funcionais, os casos de uso, o modelo de dados, as tecnologias, o fluxo operacional e os endpoints da API. O sistema utiliza HTML, CSS e JavaScript no front-end, Node.js e Express no back-end e SQLite para persistência. O resultado é uma aplicação funcional, documentada e rastreável, adequada à demonstração acadêmica.
Palavras-chave: aplicação web; gestão olímpica; controle de acesso; Node.js; SQLite.

LISTA DE ABREVIATURAS E SIGLAS

Sigla	Significado
API	Application Programming Interface
CRUD	Create, Read, Update and Delete
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
RBAC	Role-Based Access Control
REST	Representational State Transfer
SQL	Structured Query Language
UML	Unified Modeling Language

SUMÁRIO

1 INTRODUÇÃO **8**
1.1 Contextualização 8
1.2 Problema de pesquisa 8
1.3 Justificativa 8
1.4 Objetivos 8
1.4.1 Objetivo geral 8
1.4.2 Objetivos específicos 8
1.5 Organização do trabalho 9
2 LEVANTAMENTO DE REQUISITOS **9**
2.1 Atores do sistema 10
2.2 Regras de negócio 10
3 REQUISITOS FUNCIONAIS **11**
4 REQUISITOS NÃO FUNCIONAIS **12**
5 DIAGRAMA DE CASOS DE USO **13**
6 DETALHAMENTO DOS CASOS DE USO **14**
6.1 UC-01 - Autenticar no sistema 14
6.2 UC-02 - Gerenciar equipes 14
6.3 UC-03 - Gerenciar atletas 14
6.4 UC-04 - Consultar informações 15
6.5 UC-05 - Gerenciar resultados 15
6.6 UC-06 - Gerenciar usuários 15
6.7 UC-07 - Alterar configurações 16
6.8 UC-08 - Encerrar sessão 16
7 DER E MER **17**
8 TECNOLOGIAS UTILIZADAS **19**
8.1 Aplicação web e comunicação HTTP 19
8.2 Node.js e Express 19
8.3 SQLite e persistência 19
8.4 Autenticação e segurança 19
9 FLUXOGRAMA **20**
10 ENDPOINTS UTILIZADOS **22**
10.1 Códigos HTTP utilizados 23
11 METODOLOGIA E PROCESSO DE DESENVOLVIMENTO **24**
11.1 Abordagem adotada 24
11.2 Histórico de evolução 24
11.3 Processo aplicado às mudanças 25
12 IMPLEMENTAÇÃO DO SISTEMA **26**
12.1 Organização do código 26
12.2 Interface e navegação 26
12.3 Autenticação e sessões 26
12.4 Autorização 26
12.5 Decisões do projeto 27
13 RESULTADOS E EVOLUÇÃO DA INTERFACE **28**
13.1 Resultados funcionais 28
13.2 Situação inicial da navegação 28
13.3 Referência de organização visual 28

13.4 Resultado implementado 29
13.5 Síntese da evolução 30
15 CONCLUSÃO 32
15.1 Trabalhos futuros 32
REFERÊNCIAS 33
APÊNDICE A - DICIONÁRIO DE DADOS 34
Modalidades 34
Equipes 34
Atletas 34
Usuários 34
Sessões 35
Resultados 35
APÊNDICE B - MATRIZ DE RASTREABILIDADE 36
APÊNDICE C - INSTALAÇÃO E OPERAÇÃO 37
Pré-requisitos 37
Procedimento de instalação 37
Credenciais de demonstração 37
Operação e manutenção 37

1 INTRODUÇÃO

1.1 Contextualização

A organização de competições esportivas envolve informações relacionadas a modalidades, equipes, atletas, partidas e resultados. Quando esses dados são mantidos em registros separados, torna-se mais difícil localizar informações, controlar alterações e estabelecer responsabilidades entre os usuários do sistema.

O Sistema de Gestão Olímpica foi desenvolvido como uma aplicação web acadêmica para reunir essas informações em uma única interface. A aplicação é executada localmente, utiliza uma API para comunicação entre interface e servidor e aplica regras de acesso de acordo com o perfil autenticado.

1.2 Problema de pesquisa

Como desenvolver uma aplicação web capaz de centralizar informações olímpicas, manter os vínculos entre modalidades, equipes e atletas e, ao mesmo tempo, controlar as operações disponíveis para diferentes perfis de usuário?

1.3 Justificativa

O projeto permite aplicar conhecimentos de programação web, banco de dados, engenharia de requisitos, segurança e usabilidade em uma solução integrada. Sua construção também demonstra a importância de documentar as decisões tomadas, pois cada alteração na interface ou nas regras de negócio produz impactos no código, no banco, nos testes e nas permissões.

1.4 Objetivos

1.4.1 Objetivo geral

Desenvolver uma aplicação web para centralizar o cadastro e a consulta de modalidades, equipes, atletas, resultados e usuários, com autenticação e controle de acesso baseado em perfis.

1.4.2 Objetivos específicos

- Cadastrar e consultar equipes vinculadas a modalidades olímpicas.
- Cadastrar e consultar atletas vinculados a equipes.
- Registrar jogos, placares, datas, locais e situação dos resultados.
- Diferenciar as permissões dos perfis administrador, gerente e usuário.
- Permitir o gerenciamento de usuários somente pelo administrador.
- Disponibilizar configurações para alteração dos dados da própria conta.

- Organizar os módulos em uma barra lateral responsiva e recolhível.

1.5 Organização do trabalho

Após a introdução, o trabalho apresenta, nesta ordem, o levantamento de requisitos, os requisitos funcionais, os requisitos não funcionais, o diagrama e o detalhamento dos casos de uso, o DER/MER, as tecnologias utilizadas, o fluxograma e os endpoints da API. Na sequência, são registrados a metodologia, a implementação, os resultados, os testes, a conclusão e os apêndices técnicos.

2 LEVANTAMENTO DE REQUISITOS

O levantamento de requisitos foi realizado a partir das necessidades de cadastro, consulta, segurança e organização da interface. Foram identificados três perfis de acesso, as regras de negócio que limitam cada operação e os vínculos obrigatórios entre modalidades, equipes, atletas, resultados e usuários.

2.1 Atores do sistema

Quadro 1 - Atores e responsabilidades

Ator	Descrição	Responsabilidade principal
Administrador	Perfil com acesso completo.	Gerencia equipes, atletas, resultados, usuários e os dados da própria conta.
Gerente de equipes	Perfil operacional intermediário.	Cria, edita e exclui equipes; consulta modalidades, atletas e resultados.
Usuário	Perfil de consulta.	Visualiza modalidades, equipes, atletas e resultados; altera os próprios dados.

Fonte: elaboração própria (2026).

2.2 Regras de negócio

Quadro 2 - Regras de negócio

ID	Regra
RN-01	Todo acesso aos dados exige autenticação, exceto a rota de login.
RN-02	Somente administrador e gerente podem criar, editar ou excluir equipes.
RN-03	Somente o administrador pode criar, editar ou excluir atletas.
RN-04	Somente o administrador pode criar, editar ou excluir resultados.
RN-05	Somente o administrador pode acessar o painel de usuários.
RN-06	O administrador não pode excluir a própria conta pela interface administrativa.
RN-07	Uma equipe com atletas vinculados não pode ser excluída.
RN-08	Todo atleta deve pertencer a uma equipe existente.
RN-09	Toda equipe deve pertencer a uma modalidade existente.
RN-10	A alteração de senha exige a senha atual correta.
RN-11	Os logins são únicos, sem diferenciação entre letras maiúsculas e minúsculas.
RN-12	As sessões autenticadas expiram após 12 horas.
RN-13	As modalidades são inseridas por carga inicial e consultadas pela interface.

Fonte: elaboração própria (2026).

3 REQUISITOS FUNCIONAIS

Os requisitos funcionais descrevem as operações que o sistema deve disponibilizar aos usuários, respeitando as permissões definidas para cada perfil.

Quadro 3 - Requisitos funcionais

ID	Requisito	Prioridade
RF-01	Autenticar usuário por login e senha.	Alta
RF-02	Encerrar a sessão do usuário.	Alta
RF-03	Listar modalidades olímpicas.	Alta
RF-04	Consultar equipes e modalidades.	Alta
RF-05	Gerenciar equipes conforme o perfil.	Alta
RF-06	Consultar atletas e seus vínculos.	Alta
RF-07	Gerenciar atletas como administrador.	Alta
RF-08	Consultar resultados dos jogos.	Alta
RF-09	Gerenciar resultados como administrador.	Alta
RF-10	Gerenciar usuários como administrador.	Alta
RF-11	Alterar os dados da própria conta.	Média
RF-12	Adaptar a navegação a telas menores.	Média
RF-13	Recolher e expandir o menu lateral.	Baixa
RF-14	Exibir indicadores do sistema.	Média

Fonte: elaboração própria (2026).

4 REQUISITOS NÃO FUNCIONAIS

Os requisitos não funcionais estabelecem critérios de qualidade, segurança, compatibilidade, integridade, usabilidade e manutenção da aplicação.

Quadro 4 - Requisitos não funcionais

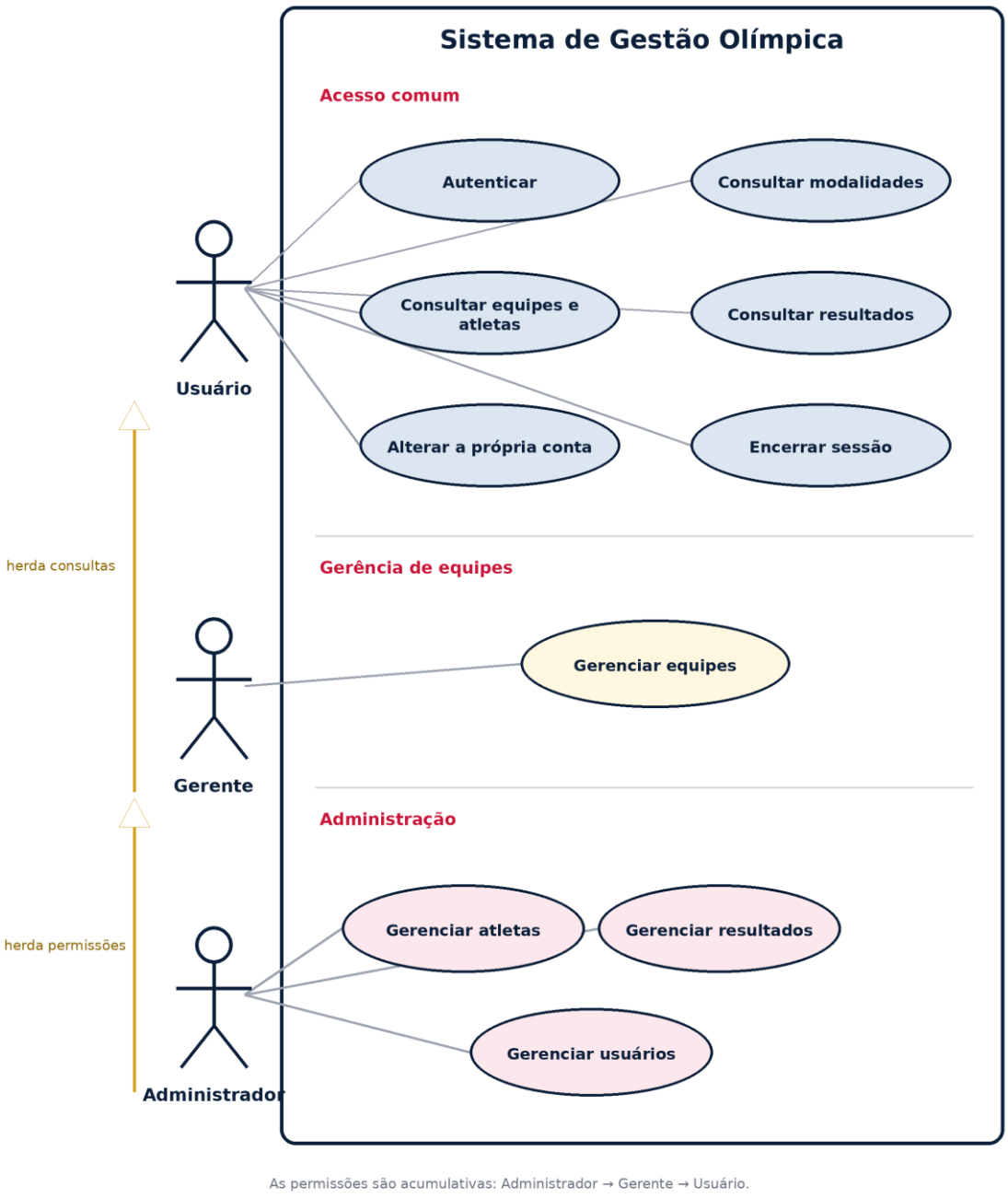
ID	Categoria	Requisito
RNF-01	Segurança	Não armazenar senhas em texto puro.
RNF-02	Segurança	Aplicar autorização no servidor.
RNF-03	Usabilidade	Apresentar mensagens claras de sucesso e erro.
RNF-04	Responsividade	Manter a navegação utilizável em computador e celular.
RNF-05	Manutenibilidade	Separar interface, servidor e inicialização do banco.
RNF-06	Compatibilidade	Executar em Node.js 18 ou superior, com recomendação da versão 22.
RNF-07	Integridade	Aplicar chaves estrangeiras e restrições no SQLite.
RNF-08	Rastreabilidade	Manter documentação e histórico sincronizados com o código.

Fonte: elaboração própria (2026).

5 DIAGRAMA DE CASOS DE USO

O diagrama apresenta os atores, as funcionalidades acessíveis e a herança de permissões. O usuário consulta informações e administra os próprios dados; o gerente acrescenta o gerenciamento de equipes; e o administrador acumula essas permissões e gerencia atletas, resultados e usuários.

Figura 1 - Diagrama de casos de uso do sistema



Fonte: elaboração própria (2026).

6 DETALHAMENTO DOS CASOS DE USO

Os casos de uso a seguir registram o objetivo, os atores, as condições, o fluxo principal, as exceções e o resultado esperado de cada operação.

6.1 UC-01 - Autenticar no sistema

Campo	Descrição
Atores	Administrador, gerente de equipes e usuário.
Objetivo	Validar as credenciais e liberar a interface conforme o perfil.
Pré-condições	Usuário ativo e previamente cadastrado.
Fluxo principal	1. Informar login e senha. 2. Validar os campos. 3. Conferir o hash da senha. 4. Criar uma sessão com validade de 12 horas. 5. Carregar o perfil e a interface autorizada.
Exceções	Campos vazios, credenciais inválidas, usuário inativo ou sessão não criada.
Pós-condições	Sessão válida e perfil disponível para as verificações de autorização.

6.2 UC-02 - Gerenciar equipes

Campo	Descrição
Atores	Administrador e gerente de equipes.
Objetivo	Cadastrar, alterar e excluir equipes vinculadas a modalidades.
Pré-condições	Sessão válida, perfil permitido e modalidade existente.
Fluxo principal	1. Abrir Raia 01. 2. Informar nome, país, modalidade e técnico. 3. Salvar a equipe. 4. Atualizar a listagem. 5. Editar ou excluir quando necessário.
Exceções	Modalidade inexistente, campos inválidos ou equipe com atletas vinculados.
Pós-condições	Equipe persistida e apresentada na listagem.

6.3 UC-03 - Gerenciar atletas

Campo	Descrição
Atores	Administrador.
Objetivo	Cadastrar, alterar e excluir atletas vinculados a uma equipe.
Pré-condições	Administrador autenticado e equipe existente.
Fluxo principal	1. Abrir Raia 02. 2. Informar os dados do atleta. 3. Selecionar a equipe. 4. Salvar. 5. Editar ou excluir quando necessário.
Exceções	Equipe inexistente, dados inválidos ou perfil sem permissão.
Pós-condições	Atleta persistido e relacionado à equipe selecionada.

6.4 UC-04 - Consultar informações

Campo	Descrição
Atores	Todos os perfis autenticados.
Objetivo	Visualizar modalidades, equipes, atletas, resultados e indicadores.
Pré-condições	Sessão válida.
Fluxo principal	1. Selecionar o módulo na barra lateral. 2. Solicitar os dados à API. 3. Pesquisar ou filtrar a listagem. 4. Visualizar os detalhes permitidos.
Exceções	Sessão expirada, falha de comunicação ou registro não encontrado.
Pós-condições	Informações exibidas sem alteração dos registros.

6.5 UC-05 - Gerenciar resultados

Campo	Descrição
Atores	Administrador.
Objetivo	Registrar e manter os resultados dos jogos.
Pré-condições	Administrador autenticado e modalidade existente.
Fluxo principal	1. Abrir Raia 03. 2. Informar equipes, placares, data, local e situação. 3. Validar os dados. 4. Salvar. 5. Editar ou excluir quando necessário.
Exceções	Dados inválidos, modalidade inexistente ou perfil sem permissão.
Pós-condições	Resultado disponível para consulta.

6.6 UC-06 - Gerenciar usuários

Campo	Descrição
Atores	Administrador.
Objetivo	Criar, editar, ativar, desativar e excluir contas do sistema.
Pré-condições	Administrador autenticado.
Fluxo principal	1. Abrir o Painel de usuários. 2. Informar nome, login, senha, perfil e estado. 3. Salvar a conta. 4. Editar ou excluir outro usuário quando necessário.
Exceções	Login duplicado, senha insuficiente ou tentativa de excluir a própria conta.
Pós-condições	Conta criada, atualizada ou removida conforme a operação.

6.7 UC-07 - Alterar configurações

Campo	Descrição
Atores	Todos os perfis autenticados.
Objetivo	Atualizar nome, login e senha da própria conta.
Pré-condições	Sessão válida.
Fluxo principal	1. Abrir Configurações. 2. Alterar nome ou login. 3. Para mudar a senha, informar a senha atual e a nova senha. 4. Confirmar e salvar.
Exceções	Login duplicado, senha atual incorreta ou confirmação divergente.
Pós-condições	Dados pessoais atualizados.

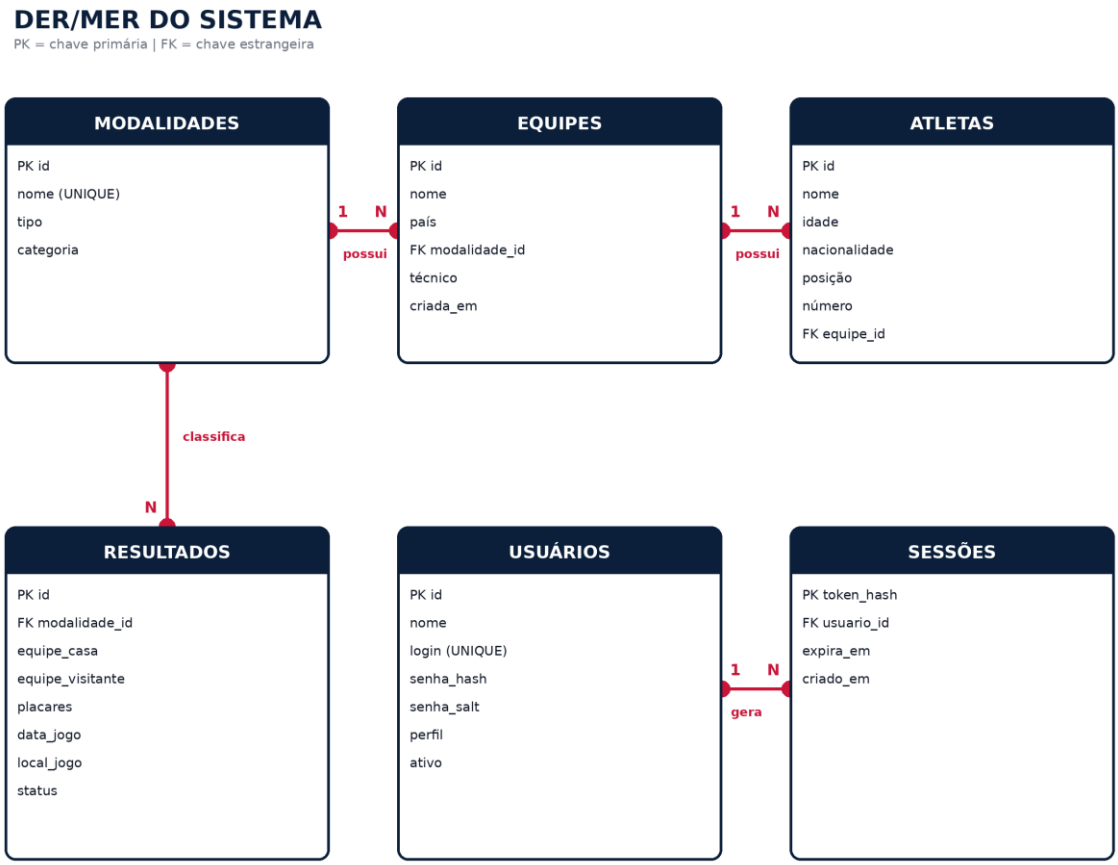
6.8 UC-08 - Encerrar sessão

Campo	Descrição
Atores	Todos os perfis autenticados.
Objetivo	Finalizar o acesso autenticado com segurança.
Pré-condições	Sessão válida.
Fluxo principal	1. Selecionar Sair. 2. Solicitar a remoção da sessão no servidor. 3. Apagar o token local. 4. Retornar à tela de login.
Exceções	Em falha de rede, remover o token local e bloquear o acesso à interface.
Pós-condições	Sessão encerrada e tela de login exibida.

7 DER E MER

O Modelo Entidade-Relacionamento define as entidades persistidas e o Diagrama Entidade-Relacionamento representa visualmente seus vínculos. A cardinalidade 1:N indica que um registro da entidade de origem pode se relacionar com vários registros da entidade de destino; no sentido inverso, a relação é N:1.

Figura 2 - DER/MER do banco de dados



Fonte: elaboração própria (2026).

APÊNDICE A - DICIONÁRIO DE DADOS

Modalidades

Campo	Tipo	Descrição
id	INTEGER PK	Identificador
nome	TEXT UNIQUE	Nome da modalidade
tipo	TEXT	Individual ou coletiva
categoria	TEXT	Verão ou Inverno

Equipes

Campo	Tipo	Descrição
id	INTEGER PK	Identificador
nome	TEXT	Nome da equipe
pais	TEXT	País representado
modalidade_id	INTEGER FK	Modalidade vinculada
tecnico	TEXT	Técnico opcional
criada_em	TEXT	Data de criação

Atletas

Campo	Tipo	Descrição
id	INTEGER PK	Identificador
nome	TEXT	Nome do atleta
idade	INTEGER	Idade
nacionalidade	TEXT	Nacionalidade
posicao	TEXT	Posição opcional
numero	INTEGER	Número opcional
equipe_id	INTEGER FK	Equipe vinculada
criado_em	TEXT	Data de criação

Usuários

Campo	Tipo	Descrição
id	INTEGER PK	Identificador
nome	TEXT	Nome de exibição
login	TEXT UNIQUE	Login único

senha_hash	TEXT	Hash scrypt
senha_salt	TEXT	Salt individual
perfil	TEXT	administrador, gerente ou usuario
ativo	INTEGER	1 ativo; 0 inativo
criado_em	TEXT	Data de criação

Sessões

Campo	Tipo	Descrição
token_hash	TEXT PK	SHA-256 do token
usuario_id	INTEGER FK	Usuário autenticado
expira_em	TEXT	Expiração da sessão
criado_em	TEXT	Data de criação

Resultados

Campo	Tipo	Descrição
id	INTEGER PK	Identificador
modalidade_id	INTEGER FK	Modalidade
equipe_casa	TEXT	Equipe A
equipe_visitante	TEXT	Equipe B
placar_casa	INTEGER	Placar A
placar_visitante	INTEGER	Placar B
data_jogo	TEXT	Data e hora
local_jogo	TEXT	Local opcional
status	TEXT	Situação do jogo
criado_em	TEXT	Data de criação

8 TECNOLOGIAS UTILIZADAS

8.1 Aplicação web e comunicação HTTP

O front-end envia requisições HTTP à API e recebe respostas em JSON. As rotas seguem uma organização REST para consulta, criação, alteração e exclusão dos recursos.

8.2 Node.js e Express

O Node.js executa JavaScript no servidor, e o Express organiza as rotas, as regras de negócio, a autenticação, a autorização e a disponibilização dos arquivos estáticos.

8.3 SQLite e persistência

O SQLite mantém o banco em um arquivo local. A biblioteca better-sqlite3 realiza o acesso aos dados e aplica chaves estrangeiras para preservar os vínculos entre as entidades.

8.4 Autenticação e segurança

As senhas são derivadas com scrypt e salt individual. As sessões utilizam tokens aleatórios armazenados como hash SHA-256, e o RBAC controla as permissões dos perfis administrador, gerente e usuário.

Quadro 6 - Tecnologias e responsabilidades

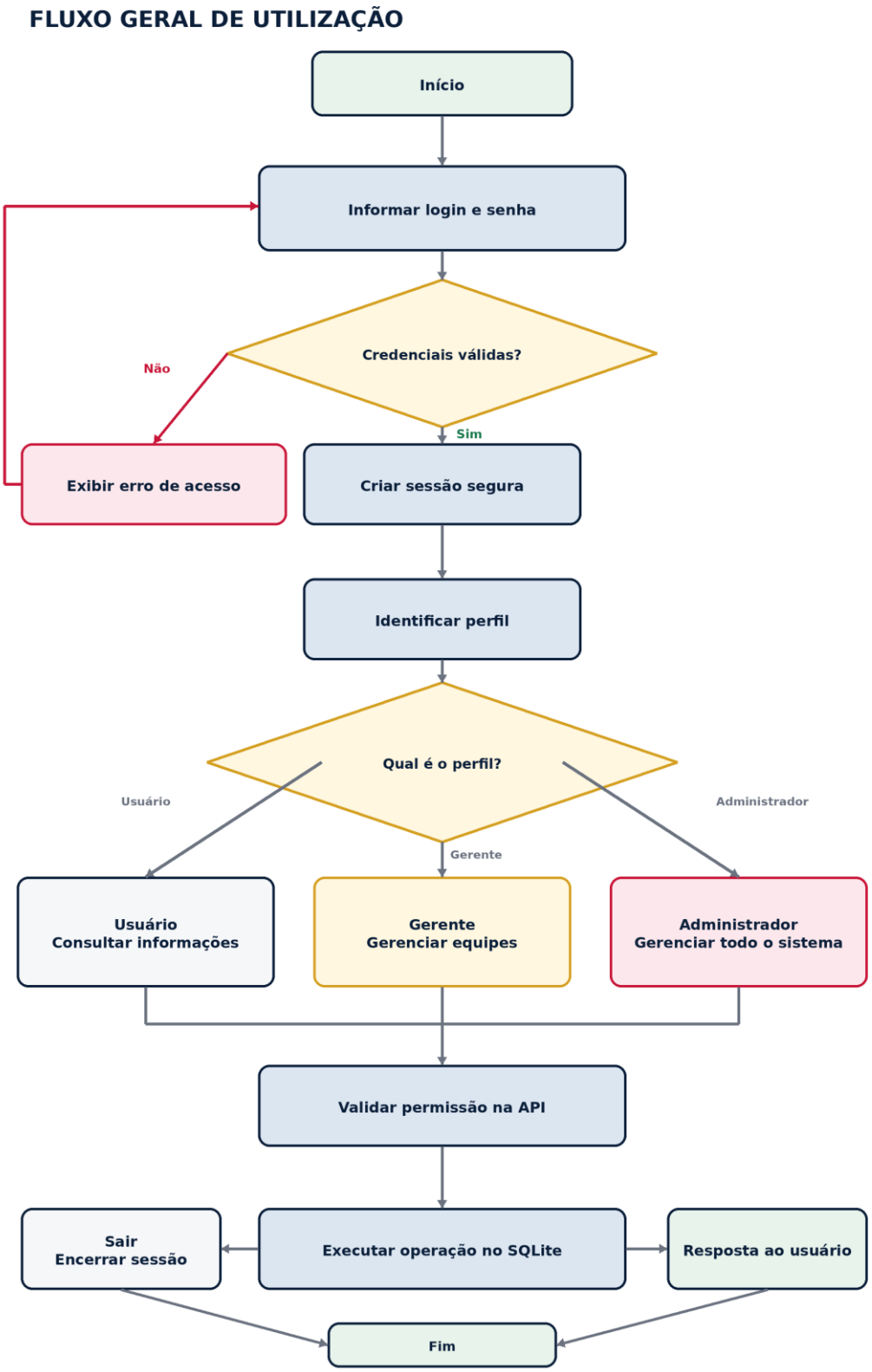
Camada	Tecnologia	Responsabilidade
Front-end	HTML5, CSS3 e JavaScript	Telas, formulários, navegação, validações e consumo da API.
Back-end	Node.js 22 e Express	Rotas HTTP, regras de negócio, autenticação e autorização.
Banco de dados	SQLite e better-sqlite3	Persistência, consultas e manutenção dos relacionamentos.
Integração	HTTP, REST e JSON	Comunicação padronizada entre navegador e servidor.
Segurança	crypto, scrypt e SHA-256	Proteção de senhas, tokens e sessões.

Fonte: elaboração própria (2026).

9 FLUXOGRAMA

O fluxo começa com a autenticação. Após a criação da sessão, o servidor identifica o perfil, verifica a permissão da operação, executa a ação no banco de dados e devolve a resposta à interface. Ao selecionar Sair, a sessão é invalidada e o acesso retorna à tela de login.

Figura 3 - Fluxograma geral de utilização do sistema



Fonte: elaboração própria (2026).

10 ENDPOINTS UTILIZADOS

A API utiliza a base local http://localhost:3000/api. Somente POST /auth/login é público; as demais rotas exigem token Bearer e verificam o perfil quando a operação altera dados.

Quadro 7 - Endpoints da API

Método	Rota	Acesso	Finalidade
POST	/auth/login	Público	Validar credenciais e criar sessão.
GET	/auth/me	Autenticado	Retornar o usuário atual.
POST	/auth/logout	Autenticado	Excluir a sessão atual.
PUT	/configuracoes	Autenticado	Alterar os próprios dados.
GET	/modalidades	Autenticado	Listar modalidades.
GET	/equipes	Autenticado	Listar equipes.
POST	/equipes	Admin/Gerente	Criar equipe.
PUT	/equipes/:id	Admin/Gerente	Atualizar equipe.
DELETE	/equipes/:id	Admin/Gerente	Excluir equipe sem atletas.
GET	/atletas	Autenticado	Listar atletas.
POST	/atletas	Administrador	Criar atleta.
PUT	/atletas/:id	Administrador	Atualizar atleta.
DELETE	/atletas/:id	Administrador	Excluir atleta.
GET	/resultados	Autenticado	Listar resultados.
POST	/resultados	Administrador	Criar resultado.
PUT	/resultados/:id	Administrador	Atualizar resultado.
DELETE	/resultados/:id	Administrador	Excluir resultado.
GET	/usuarios	Administrador	Listar usuários.
POST	/usuarios	Administrador	Criar usuário.
PUT	/usuarios/:id	Administrador	Editar usuário.
DELETE	/usuarios/:id	Administrador	Excluir outro usuário.

Fonte: elaboração própria (2026).

10.1 Códigos HTTP utilizados

Quadro 8 - Códigos HTTP utilizados

Código	Significado
200	Consulta ou atualização concluída.
201	Registro criado.
204	Operação concluída sem corpo de resposta.
400	Dados inválidos ou regra de negócio violada.
401	Credenciais inválidas ou sessão expirada.
403	Perfil sem permissão.
404	Registro não encontrado.
409	Conflito, como login já utilizado.
500	Erro interno não previsto.

Fonte: elaboração própria (2026).

11 METODOLOGIA E PROCESSO DE DESENVOLVIMENTO

11.1 Abordagem adotada

O desenvolvimento ocorreu de forma incremental. A cada solicitação, foram identificados os componentes afetados, implementadas as alterações, executados testes e atualizada a documentação. Essa abordagem permitiu preservar funcionalidades existentes enquanto novos módulos eram incorporados.

11.2 Histórico de evolução

Quadro 9 - Evolução do projeto

Etapa	Período	Alterações principais
Base inicial	21/08/2026	Cadastro de equipes e atletas, modalidades iniciais, API Express e banco SQLite.
Ambiente	21 a 28/08/2026	Correção da instalação, adoção do Node.js 22 e configuração no IntelliJ IDEA.
Terminologia	28/08/2026	Padronização integral do termo atleta na interface, API, variáveis e banco.
Autenticação	28/08/2026	Tela de login, sessões e perfis administrador, gerente e usuário.
Novos módulos	28/08/2026	Resultados de jogos e painel administrativo de usuários.
Interface lateral	28/08/2026	Migração das raias para menu lateral recolhível e responsivo.
Configurações	28/08/2026	Alteração de nome, login e senha da própria conta.
Documentação	18/09/2026	Consolidação acadêmica, técnica e visual do projeto.

Fonte: elaboração própria (2026).

11.3 Processo aplicado às mudanças

Quadro 10 - Etapas para implementação de mudanças

Etapa	Atividade
1. Solicitação	Definir o problema, o objetivo, o perfil afetado e o resultado esperado.
2. Análise de impacto	Identificar telas, rotas, tabelas, permissões e documentos afetados.
3. Implementação	Alterar o código e preservar as regras existentes.
4. Verificação	Executar testes técnicos, funcionais e de permissão.
5. Documentação	Atualizar requisitos, casos de uso, API, banco e rastreabilidade.
6. Entrega	Gerar uma nova versão do projeto e da documentação.

Fonte: elaboração própria (2026).

12 IMPLEMENTAÇÃO DO SISTEMA

12.1 Organização do código

A estrutura do projeto separa o back-end, o front-end e a documentação. O arquivo `database.js` inicializa o banco e os dados de demonstração; `server.js` concentra a API, a autenticação e as regras de autorização; e os arquivos do front-end definem a interface, os estilos e as interações.

```
olimpiadas-cadastro/
├── backend/
│   ├── database.js
│   ├── server.js
│   └── package.json
├── frontend/
│   ├── index.html
│   ├── css/style.css
│   └── js/script.js
└── README.md
```

12.2 Interface e navegação

A tela de login apresenta a identidade visual do projeto e solicita login e senha. Após a autenticação, a barra lateral organiza os módulos como Raia 01 - Equipes, Raia 02 - Atletas, Raia 03 - Resultados e Painel Usuários. A área de Configurações permite que o usuário altere os próprios dados.

Quadro 11 - Módulos e disponibilidade

Módulo	Finalidade	Disponibilidade
Raia 01 - Equipes	Cadastro e consulta de equipes.	Todos consultam; administrador e gerente alteram.
Raia 02 - Atletas	Cadastro e consulta de atletas.	Todos consultam; administrador altera.
Raia 03 - Resultados	Jogos e placares.	Todos consultam; administrador altera.
Painel - Usuários	Administração de contas.	Somente administrador.
Configurações	Dados da própria conta.	Todos os perfis.

Fonte: elaboração própria (2026).

12.3 Autenticação e sessões

A senha é derivada com scrypt e salt aleatório. No login, a comparação utiliza uma função de tempo constante. Após a validação, o servidor cria um token aleatório e armazena apenas o seu hash SHA-256. A sessão expira depois de 12 horas ou é removida no encerramento da sessão.

12.4 Autorização

As permissões são verificadas no servidor antes das operações de escrita. A ocultação de botões no front-end melhora a experiência, mas não substitui a validação no back-end. Quando o perfil não possui acesso, a API retorna o código HTTP 403.

Quadro 12 - Matriz de permissões

Operação	Administrador	Gerente	Usuário
Consultar dados gerais	Sim	Sim	Sim
Gerenciar equipes	Sim	Sim	Não
Gerenciar atletas	Sim	Não	Não
Gerenciar resultados	Sim	Não	Não
Gerenciar usuários	Sim	Não	Não
Alterar a própria conta	Sim	Sim	Sim

Fonte: elaboração própria (2026).

12.5 Decisões do projeto

Quadro 13 - Problemas, decisões e benefícios

Problema	Decisão	Benefício
A terminologia inicial não abrangia todas as modalidades.	Padronizar o termo atleta.	Terminologia mais adequada ao contexto olímpico.
Todos os usuários tinham o mesmo acesso.	Criar três perfis.	Separação de responsabilidades e maior segurança.
A navegação horizontal ocupava espaço.	Criar barra lateral recolhível.	Melhor organização e adaptação a diferentes telas.
As senhas precisavam de proteção.	Aplicar scrypt com salt.	Senhas não armazenadas em texto puro.
A demonstração deveria ser simples.	Utilizar SQLite local.	Execução sem servidor externo de banco de dados.
Equipes precisavam de classificação.	Vincular equipes às modalidades.	Organização consistente dos dados esportivos.

Fonte: elaboração própria (2026).

13 RESULTADOS E EVOLUÇÃO DA INTERFACE

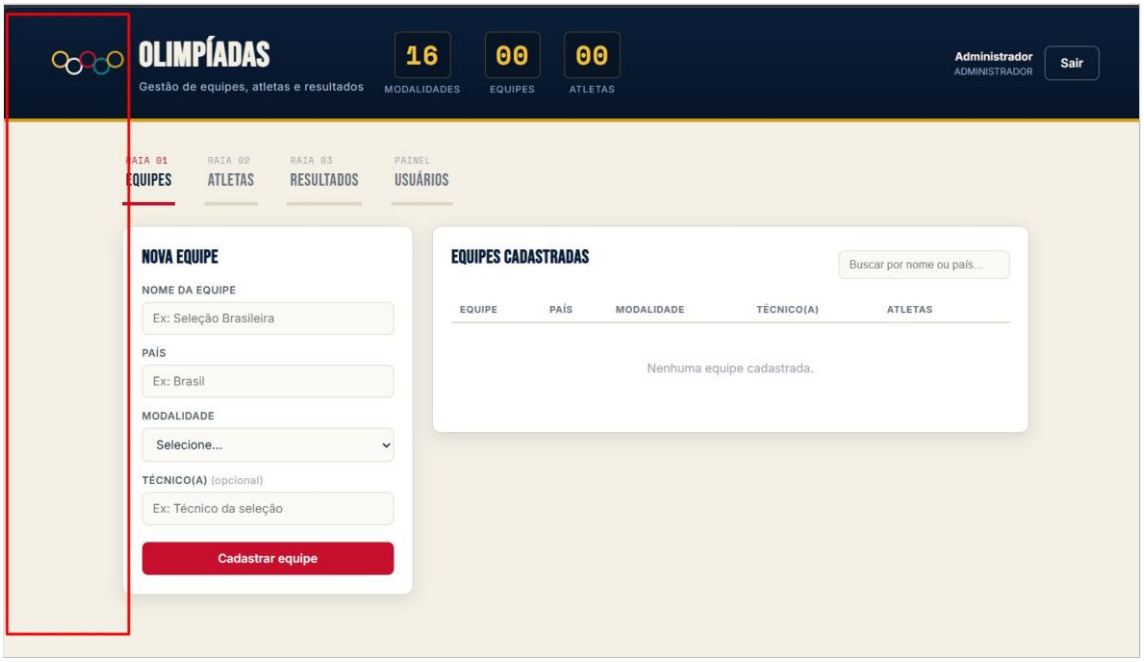
13.1 Resultados funcionais

Ao final da implementação, o sistema passou a oferecer autenticação, controle de acesso por perfil, cadastro de equipes e atletas, consulta de modalidades, registro de resultados, gerenciamento administrativo de usuários e configurações pessoais. A aplicação mantém os dados em SQLite e pode ser executada localmente com comandos npm.

13.2 Situação inicial da navegação

Na versão anterior, as raias eram exibidas horizontalmente acima do conteúdo. A área à esquerda ainda não possuía função de navegação.

Figura 4 - Interface anterior e área destinada à navegação lateral

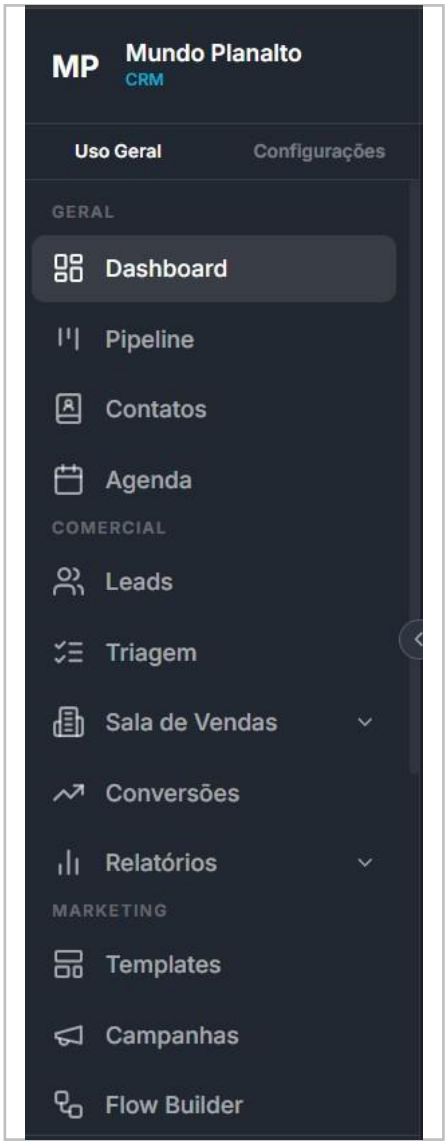


Fonte: arquivo do projeto (2026).

13.3 Referência de organização visual

A referência selecionada apresentou um menu vertical com identificação, separação de áreas, ícones e destaque do item ativo. Esses elementos foram adaptados à identidade do sistema olímpico.

Figura 5 - Referência utilizada para a organização do menu lateral

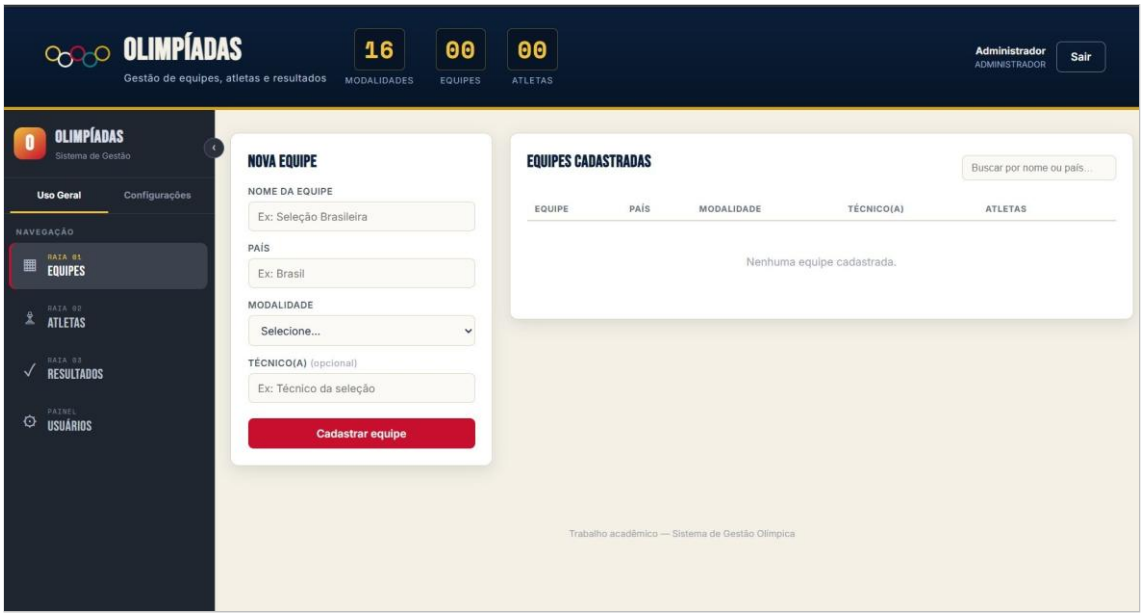


Fonte: imagem de referência fornecida pelo autor (2026).

13.4 Resultado implementado

A navegação foi posicionada à esquerda, com os módulos identificados como raias. O item selecionado recebe destaque; o menu pode ser recolhido; e o conteúdo principal se ajusta ao espaço disponível.

Figura 6 - Barra lateral implementada no sistema



Fonte: captura de tela do sistema desenvolvido (2026).

13.5 Síntese da evolução

Quadro 14 - Evolução da barra lateral

Etapa	Descrição
1. Estrutura anterior	Raias localizadas horizontalmente acima do conteúdo.
2. Referência visual	Análise de um menu vertical com áreas, ícones e destaque do item ativo.
3. Resultado final	Raias movidas para a lateral, com recolhimento, responsividade e acesso funcional.

Fonte: elaboração própria (2026).

CONCLUSÃO

O desenvolvimento do Sistema de Gestão Olímpica atendeu ao objetivo de centralizar modalidades, equipes, atletas, resultados e usuários em uma aplicação web. A separação por perfis tornou as permissões mais coerentes com as responsabilidades dos usuários, enquanto a proteção das senhas e das sessões acrescentou controles de segurança compatíveis com o escopo acadêmico.

A evolução da interface também contribuiu para a usabilidade. A migração das raias horizontais para uma barra lateral permitiu melhor aproveitamento do espaço e organização dos módulos. A comparação entre a versão inicial, a referência utilizada e a solução implementada evidencia a aplicação de decisões de projeto baseadas em problemas observados.

Além do produto funcional, o trabalho resultou em uma documentação que relaciona requisitos, regras de negócio, casos de uso, arquitetura, banco de dados, rotas e testes. Essa rastreabilidade facilita a manutenção e reduz o risco de divergência entre o código e a documentação.

13.6 Trabalhos futuros

- Implementar cadastro administrativo de modalidades.
- Criar histórico de auditoria das ações realizadas pelos usuários.
- Exportar equipes, atletas e resultados em PDF ou planilha.
- Adicionar recuperação de senha e bloqueio após tentativas inválidas.
- Criar testes automatizados para API e interface.
- Publicar o sistema em ambiente seguro com HTTPS e política de backup.

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. ABNT NBR 14724:2024: informação e documentação - trabalhos acadêmicos apresentação. Versão corrigida: 2025. Rio de Janeiro: ABNT, 2025.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. ABNT NBR

6023:2025: informação e documentação - referências - elaboração. Rio de Janeiro: ABNT, 2025.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. ABNT NBR

6024:2012: informação e documentação - numeração progressiva das seções de um documento escrito - apresentação. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. ABNT NBR

6027:2012: informação e documentação - sumário - apresentação. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. ABNT NBR

6028:2021: informação e documentação - resumo, resenha e recensão apresentação. Rio de Janeiro: ABNT, 2021.

EXPRESS. Express: fast, unopinionated, minimalist web framework for Node.js. Disponível em: <https://expressjs.com/>. Acesso em: 18 set. 2026.

NODE.JS. Introduction to Node.js. Disponível em:

<https://nodejs.org/en/learn/getting-started/introduction-to-nodejs>. Acesso em: 18 set. 2026.

SQLITE. About SQLite. Disponível em:

<https://www.sqlite.org/about.html>. Acesso em: 18 set. 2026.

APÊNDICE A - DICIONÁRIO DE DADOS

Modalidades

Campo	Tipo	Descrição
id	INTEGER PK	Identificador
nome	TEXT UNIQUE	Nome da modalidade
tipo	TEXT	Individual ou coletiva
categoria	TEXT	Verão ou Inverno

Equipes

Campo	Tipo	Descrição
id	INTEGER PK	Identificador
nome	TEXT	Nome da equipe
pais	TEXT	País representado
modalidade_id	INTEGER FK	Modalidade vinculada
tecnico	TEXT	Técnico opcional
criada_em	TEXT	Data de criação

Atletas

Campo	Tipo	Descrição
id	INTEGER PK	Identificador
nome	TEXT	Nome do atleta
idade	INTEGER	Idade
nacionalidade	TEXT	Nacionalidade
posicao	TEXT	Posição opcional
numero	INTEGER	Número opcional
equipe_id	INTEGER FK	Equipe vinculada
criado_em	TEXT	Data de criação

Usuários

Campo	Tipo	Descrição
id	INTEGER PK	Identificador
nome	TEXT	Nome de exibição
login	TEXT UNIQUE	Login único

senha_hash	TEXT	Hash scrypt
senha_salt	TEXT	Salt individual
perfil	TEXT	administrador, gerente ou usuario
ativo	INTEGER	1 ativo; 0 inativo
criado_em	TEXT	Data de criação

Sessões

Campo	Tipo	Descrição
token_hash	TEXT PK	SHA-256 do token
usuario_id	INTEGER FK	Usuário autenticado
expira_em	TEXT	Expiração da sessão
criado_em	TEXT	Data de criação

Resultados

Campo	Tipo	Descrição
id	INTEGER PK	Identificador
modalidade_id	INTEGER FK	Modalidade
equipe_casa	TEXT	Equipe A
equipe_visitante	TEXT	Equipe B
placar_casa	INTEGER	Placar A
placar_visitante	INTEGER	Placar B
data_jogo	TEXT	Data e hora
local_jogo	TEXT	Local opcional
status	TEXT	Situação do jogo
criado_em	TEXT	Data de criação

APÊNDICE B - MATRIZ DE RASTREABILIDADE

A matriz relaciona cada requisito às telas, rotas, tabelas e arquivos principais. Antes de alterar uma funcionalidade, os elementos da mesma linha devem ser revisados.

Requisit o	Tela	Rota	Tabela	Arquivo
RF-01	Login	POST /auth/login	usuarios, sessoes	server.js; script.js
RF-02	Sair	POST /auth/logout	sessoes	server.js; script.js
RF-03	Indicadores	GET /modalidades	modalidades	database.js; server.js
RF-04/05	Raia 01	CRUD /equipes	equipes	index.html; server.js; script.js
RF-06/07	Raia 02	CRUD /atletas	atletas	index.html; server.js; script.js
RF-08/09	Raia 03	CRUD /resultados	resultados	index.html; server.js; script.js
RF-10	Painel	CRUD /usuarios	usuarios	index.html; server.js; script.js
RF-11	Configurações	PUT /configuracoes	usuarios	index.html; server.js; script.js
RF-12/13	Menu lateral	Não se aplica	Não se aplica	style.css; script.js
RF-14	Cabeçalho	GET de três recursos	3 tabelas	index.html; script.js

APÊNDICE C - INSTALAÇÃO E OPERAÇÃO

Pré-requisitos

- Windows, Linux ou macOS.
- Node.js 18 ou superior; recomenda-se Node.js 22.
- npm disponível no terminal.

Procedimento de instalação

Extraia o projeto, abra o terminal na pasta `olimpiadas-cadastro/backend` e execute os comandos:

```
npm install
npm start
```

Após a mensagem de inicialização, acesse `http://localhost:3000` no navegador.

Credenciais de demonstração

Perfil	Login	Senha inicial
Administrador	admin	admin123
Gerente	gerente	gerente123
Usuário	usuario	usuario123

Observação: as credenciais iniciais são destinadas apenas à demonstração acadêmica e devem ser alteradas antes de qualquer uso real.

Operação e manutenção

O comando `npm start` inicia o servidor, enquanto `npm run dev` reinicia a aplicação quando há mudanças no código. O banco `backend/olimpiadas.db` é criado automaticamente. Recomenda-se realizar backup antes de substituir ou remover esse arquivo.